School ERP System

Technical Documentation & Team Delivery Plan — All 14 Modules

Scope: Full product, phased delivery · Team size: 2 engineers

Frontend: React (TypeScript) · Backend: NestJS (TypeScript) · DB: PostgreSQL (multi-tenant)

Table of Contents

Table of Contents 2
1. Objective of This Document 3
2. Framework Decision & Rationale 3
2.1 Frontend — React (fixed) 3
2.2 Backend — NestJS (recommended) 3
2.3 Database — PostgreSQL 4
2.4 Supporting Services 4
2.5 Multi-Tenancy — One Codebase, Multiple Schools 5
2.6 Hosting & the AWS Migration Path 5
3. Full Product Scope — All 14 Modules 6
4. Team Structure 7
5. Phase-by-Phase Deliverables 7
Phase 0 — Foundations (Week 1) 8
Phase 1 — Admissions Core Workflow (Weeks 2–5) 8
Phase 2 — SIS, Communication & Fee Closure (Weeks 6–7) 9
Interim Milestone — Admissions Checkpoint (Week 8) 9
Phase 3 — Attendance & Timetable with Substitution (Weeks 9–11) 9
Phase 4 — Leave / Gate Pass (Weeks 12–13) 10
Phase 5 — Discipline & Safeguarding (Weeks 14–15) 10
Phase 6 — Academics (Weeks 16–18) 11
Phase 7 — Reports & Dashboards (Weeks 19–20) 11
Phase 8 — Hostel Management (Weeks 21–22) 11
Phase 9 — Staff Management & Payroll (Weeks 23–25) 11
Phase 10 — CBT: Computer-Based Testing (Weeks 26–27) 12
Phase 11 — Final QA, Integration & Deployment (Week 28) 12
6. Full Timeline Summary 12
7. Appendix — Shared Status Enum (Reference for Both Engineers) 13

1. Objective of This Document

This document defines the technology stack for the School ERP system, explains why each choice was made over comparable alternatives, sequences all 14 modules into delivery phases, and assigns concrete, phase-by-phase deliverables to each of the two engineers on the team. It is meant to be the single reference both engineers and the reviewer (you) use to track who owns what, and when.

> **Team assumption**
>
> Two engineers: one owns the entire frontend (React application, all screens, all roles), one owns the entire backend (API, database, integrations, jobs, deployment). Adjust names/assignments below as needed — the task breakdown stays the same regardless of who is assigned to which side.

2. Framework Decision & Rationale

The workflow you provided (Sections 1–9 of the Admissions flow) is not a simple form-and-database app. It has 6+ distinct roles, ~9 sequential approval states with branching (Valid/Invalid, Approve/Reject), scheduled reminders, external payment and messaging integrations, and document handling. The stack below is chosen specifically to handle that complexity cleanly, not just to get an MVP out fast.

2.1 Frontend — React (fixed)

React was specified as the required framework. Within React, the following supporting choices are made to fit this specific problem:

- Vite + TypeScript — faster dev/build cycle than CRA (which is deprecated); TypeScript is not optional here because the app has ~10 status values and 6+ roles — a typo in a status string becomes a silent workflow bug, not a compile error, unless it's typed.
- React Hook Form + Zod — the Admission Form's fields are dynamic (AO can add/delete mandatory fields per academic year), so the form and its validation schema both need to be data-driven rather than hardcoded.
- TanStack Query — every dashboard in this system (AO, Accounts, OS, VP, Principal) is really the same pattern: fetch a filtered list, mutate a status, refetch. TanStack Query handles caching, refetching, and loading/error states for that pattern without hand-rolled state management.
- shadcn/ui + Tailwind CSS — pre-built accessible components (tables, dialogs, forms) that are fully customizable, so 6 role-based dashboards can be built quickly without a heavy design system.
- React Router — role-based route protection (a Parent should never reach the Principal's approval queue, etc.).
- Zustand (lightweight) — only for auth/session/role context; avoids the boilerplate of Redux for a 2-person team.

2.2 Backend — NestJS (recommended)

This is the one open decision on the stack, so it's laid out as a comparison rather than an assertion.

Criterion	NestJS (Node + TypeScript)	Django + DRF (Python)
Language alignment	Same language as React frontend — types/interfaces (e.g. AdmissionApplication, ApprovalStatus) can be shared between frontend and backend, cutting a whole class of contract-mismatch bugs.	Different language from frontend — no type sharing; contract drift between API and UI has to be caught manually or via a separate schema (OpenAPI/Swagger).

Module structure	Built-in module system maps ~1:1 to your 14 planned modules (Admissions, SIS, Fees, Hostel...) — each becomes a self-contained Nest module.	Django apps serve the same purpose and are equally modular — this is roughly a tie.
Role-based access control	Needs to be built with guards/decorators (CASL library recommended) — more setup work up front.	Django's permissions framework is built in — less setup work for RBAC specifically.
Approval / status workflow	No built-in state machine — use the xstate library or a hand-rolled status-transition service.	django-fsm is a close match for exactly this approve/reject-with-remarks chain pattern.
Background jobs (reminders, PDF gen)	BullMQ (Redis-backed) — well-suited to scheduled reminders (Day 7/13) and async PDF generation.	Celery + Redis — mature, equally capable, more moving parts to configure initially.
1-backend-person velocity	Best if the backend engineer is already comfortable in TypeScript — no context-switch from the frontend's language.	Best if the backend engineer is stronger in Python — Django's built-ins (admin panel, ORM, auth) reduce boilerplate for a solo backend owner.

Recommendation: NestJS, on the strength of shared TypeScript types across a 2-person team — that single benefit outweighs Django's built-in RBAC/FSM advantages, because those are one-time setup costs, while a frontend/backend contract mismatch is a recurring cost throughout the project. If your backend engineer is significantly stronger in Python than TypeScript, Django is a legitimate alternative and the rest of this document's module boundaries still apply unchanged.

2.3 Database — PostgreSQL

The data model is fundamentally relational: Student → Parent → Documents → Fee Payment → Roll Number → House, all linked by foreign keys, with status fields that gate transitions between tables (e.g. a roll number can't be generated until fee status = Approved). This is a textbook relational-integrity use case, not a document-store use case — PostgreSQL over MongoDB/Firebase.

2.4 Supporting Services

Concern	Choice & why
File storage (now)	Cloudflare R2 — Aadhaar, birth certificates, marksheets, photos. R2 speaks the same S3 API as AWS, has no egress fees (important once multiple schools are pulling documents regularly), and is materially cheaper at this stage. Never store documents in the DB or on local disk.
File storage (future)	AWS S3 — because R2 is accessed through the S3-compatible SDK from day one, moving to S3 later is a config/endpoint change, not a rewrite. See Section 2.6 for the full AWS migration note.
Caching	Redis — two jobs: (1) application-level cache for read-heavy, slow-changing data (published field templates, house-capacity counts, dashboard aggregate counts) to keep multi-tenant dashboards fast as school count grows; (2) session/rate-limit store for the parent portal login. This is the same Redis instance BullMQ already needs — one piece of infra, two uses.
SMS / Email / WhatsApp	MSG91 or Twilio (SMS/WhatsApp), SES or SendGrid (email) — abstracted behind one internal NotificationService so the provider can be swapped without touching business logic.

Payments	Razorpay or PayU — standard for Indian school ERPs, webhook-based so 'payment received' updates status automatically rather than relying on the parent's browser staying open.
Background jobs	BullMQ + Redis — Day 7/13 reminders, async PDF generation, notification retries. These must not block the request/response cycle.
PDF generation	pdf-lib or Puppeteer — for the auto-generated Admission Letter in Section 8 of the workflow.

2.5 Multi-Tenancy — One Codebase, Multiple Schools

A requirement surfaced in review: the system needs to serve multiple schools from one deployment, so a code change or bug fix applies to every school at once rather than being patched school-by-school. This is a real architectural decision, not a config toggle, so it's made explicit here.

Criterion	Shared schema + tenant_id (recommended)	Schema-per-tenant / DB-per-tenant
How it works	One database, every table carries a school_id column. Every query is automatically scoped to the logged-in user's school via a NestJS interceptor/middleware — application code never has to remember to filter manually.	Each school gets its own schema or its own database. Strongest possible data isolation between schools.
Fits a 2-person team?	Yes — one schema to migrate, one deployment to monitor, one place a bug fix lands. Directly matches the 'change once, applies to all schools' requirement.	No — every schema migration has to run per-tenant; with one backend engineer this multiplies operational work for no MVP-stage benefit.
Scales to more schools later?	Yes, comfortably into the hundreds of schools on Postgres with proper indexing on school_id. Can be split out later if one very large school needs true isolation.	Scales in isolation terms, but operational cost grows linearly with school count from day one — the wrong trade for an MVP with 2 engineers.

Recommendation: shared schema with a school_id foreign key on every tenant-scoped table (Student, Application, Payment, Document, ApprovalStep, etc.), enforced centrally by a request-scoped NestJS interceptor rather than repeated in every query. This is what makes "we changed the code and it changed for everyone" true by construction — there is only one copy of the code and one schema. School onboarding becomes a matter of inserting a School row and its admin user, not a deployment.

2.6 Hosting & the AWS Migration Path

AWS was flagged as the target cloud in future, not necessarily day one. For a 2-person team pre-revenue, running the MVP on AWS directly (ECS/RDS/ElastiCache) adds setup and cost overhead that isn't justified yet. The approach below gets you to AWS without a rewrite when the time comes:

- MVP hosting: Railway or Render for the NestJS API (managed Postgres + Redis add-ons included), Vercel or Netlify for the React frontend. Both are one engineer's worth of DevOps.
- Object storage: Cloudflare R2 from day one, via the S3 SDK — this is the one piece deliberately chosen to be AWS-compatible in advance.

- Migration trigger: once there's real multi-school traffic and a case for AWS's ecosystem (RDS read replicas, CloudFront, ECS auto-scaling), move the Postgres instance to RDS, Redis to ElastiCache, the API containers to ECS/Fargate, and repoint the S3 SDK from R2's endpoint to AWS S3's — no application code changes required for storage.

3. Full Product Scope — All 14 Modules

All 14 modules from your list are covered below, phased by dependency rather than by list order. The logic: Admissions produces the Student ID that almost everything else keys off, so it goes first. SIS, Parent Communication, and Fees are thin-but-foundational — other modules call into them. Everything after that is ordered by how tightly modules are coupled to each other (e.g. Attendance and Timetable share the same calendar/period data model, so they're built together).

Phase	Module(s)	Why this order
0	Foundations	Multi-tenant schema, auth/RBAC, routing skeleton — every module below depends on this existing first.
1	Admissions	Produces the Student ID and Student Master record that Attendance, Academics, Hostel, Fees, Discipline, etc. all reference.
2	Student Information System, Parent Communication, Fees & Payments	Thin, foundational services other modules call into (e.g. Attendance notifies parents through the same Parent Communication service Admissions already built).
3	Attendance, Timetable with Substitution	Share the same class/section/period data model — building them together avoids modeling the same calendar twice.
4	Leave / Gate Pass	Reuses the approval state-machine pattern from Admissions; also needs Attendance/Timetable context (a leave day affects attendance).
5	Discipline & Safeguarding	Needs Student Master and staff roles already in place; handled after core academic-operations modules due to its stricter access-control requirements.

6	Academics	Gradebook/report cards need Attendance and Timetable data (periods, subjects) already modeled.
7	Reports & Dashboards	Cross-cutting by definition — needs Attendance, Fees, and Academics data to exist before there's anything to report on.
8	Hostel Management	Independent of the academic modules; sequenced here mainly by business priority, can be moved earlier if hostel onboarding is urgent.
9	Staff Management, Payroll	Payroll depends on Staff Management and on Leave balances from Phase 4.
10	CBT (Computer-Based Testing)	Largely standalone; placed last as it's the least coupled to the rest of the system and lowest urgency of the remaining modules.
11	Final QA, Integration, Deployment	Full regression across every role and module, load-tested under multi-school conditions, then production deploy.

4. Team Structure

Person	Ownership
Engineer A — Frontend	Entire React application: all role-based screens (AO, OS, Parent, Admission Incharge, Accounts, VP, Principal), the dynamic admission form, all dashboards, client-side validation, and API integration against contracts defined in Phase 0.
Engineer B — Backend	Entire backend: API design, database schema, authentication/RBAC, the approval state machine, all third-party integrations (SMS/Email/WhatsApp, payment gateway, file storage), background jobs (reminders, PDF generation), and deployment.

5. Phase-by-Phase Deliverables

Each phase lists what Engineer A and Engineer B deliver in parallel, plus the point where they must sync (API contract, shared enum, etc.). Timeline is expressed in relative weeks — adjust to your actual sprint calendar.

Phase 0 — Foundations (Week 1)

	Engineer A — Frontend	Engineer B — Backend
Deliverables	Vite + React + TS project scaffolded Routing skeleton for all 6 roles + Parent portal shadcn/ui + Tailwind theme set up Auth screens (login for Parent, staff roles) — UI only, mocked responses	NestJS project scaffolded, module folders for Admissions/SIS/Fees/Notifications PostgreSQL schema v1: School (tenant), Student, Parent, Application, Document, ApprovalStep, Payment — every tenant-scoped table carries school_id Tenant-scoping interceptor so all queries auto-filter by the logged-in user's school_id (see Section 2.5) Auth module: JWT-based login for both staff roles and per-applicant parent credentials, JWT includes school_id R2 bucket + Redis instance provisioned; NotificationService and cache wrapper stubbed OpenAPI/Swagger doc published so Engineer A can build against a real contract
Joint sync point	Agree on the full status enum (Pending Approval, Published, Submitted by Parent, Document Validated, Re-submit, Payment Received, Fee Approved, Roll No Assigned, VP Approved, Admission Confirmed) and the API contract shape before either side builds against it.	

Phase 1 — Admissions Core Workflow (Weeks 2–5)

This is the largest phase and maps directly to Sections 1–8 of your workflow document.

Week	Engineer A — Frontend	Engineer B — Backend
Week 2	AO screen: Mandatory Field builder (add/delete field, save as template per academic year) Send-for-Approval action + status badge UI	Mandatory field template CRUD + versioning by academic year Approval endpoint (OS approve/reject with remarks) + status transition logic
Week 3	Dynamic Admission Form renderer (fields driven by the published template, not hardcoded) Document upload UI (Aadhaar, photo, marksheet, birth cert) with client-side type/size checks Save Draft / Submit actions	Unique Login ID + Password auto-generation per applicant on 'Create Admission Form' SMS/Email/WhatsApp trigger service (link + credentials to parent) 15-day link expiry + Day 7/Day 13 reminder jobs (BullMQ) Document storage to S3 with signed URLs
Week 4	Admission Incharge / AO dashboard: list of Submitted applications Validate Documents action (Valid / Invalid + remarks)	Document validation endpoint + Re-submit branch back to parent Payment link generation on Valid → integrate Razorpay/PayU

	Payment screen for parent (post-validation)	Webhook handler for payment confirmation
Week 5	Accounts dashboard: Payment Received list, Verify & Approve + receipt upload OS dashboard: Fee Approved list, Generate Roll No, Assign House VP dashboard: review queue, Approve / Send Back Principal dashboard: final approval + digital sign UI	Accounts approval endpoint, receipt number storage Roll number auto-generation rule: [CLASS][YEAR][SEQ] House allocation logic (random/capacity-based) VP and Principal approval endpoints with send-back branching Admission Letter PDF auto-generation on final approval

Phase 2 — SIS, Communication & Fee Closure (Weeks 6–7)

	Engineer A — Frontend	Engineer B — Backend
Deliverables	Student Master profile view (read-only) — Class/Section/House/Student ID Notification log/history view per applicant (for AO) Confirmation screen shown to parent post Admission Confirmed	Move confirmed applications into Student Master table + generate Student ID Archive all documents into a Student DMS structure (S3 folder-per-student) Notification history persisted and queryable Export hooks stubbed for TC / ID Card / Fee Module (data shape only, not full features)

Interim Milestone — Admissions Checkpoint (Week 8)

Before continuing into Attendance and beyond, close out the Admissions-scoped slice as a working checkpoint. This is not the final QA pass (that's Phase 11) — it's a smaller regression to confirm Phases 0–2 are solid before building on top of them.

	Engineer A — Frontend	Engineer B — Backend
Deliverables	End-to-end walkthrough of all 9 workflow steps across every role Error/empty/loading states for every dashboard Responsive check (staff likely on desktop, parents likely on mobile)	Load test the reminder job queue and payment webhook path Environment config for staging/production, secrets management Deploy: backend + Postgres + Redis; frontend to static hosting/CDN Basic monitoring/logging (errors, failed notification sends)
Joint sync point	Run the full admission cycle together, one application, start to finish, through every role, before calling the MVP done.	

Phase 3 — Attendance & Timetable with Substitution (Weeks 9–11)

Grouped together because both are built on the same class/section/period calendar model.

	Engineer A — Frontend	Engineer B — Backend
Attendance	Daily attendance marking screen (teacher, class-wise, bulk-mark) Per-student attendance calendar view Absentee report / low-attendance flag view	Attendance table (student, date, status, marked_by, school_id) Bulk-mark API + attendance % calculation endpoint Auto-notify parent on absence via existing Parent Communication service
Timetable	Weekly timetable grid per class and per teacher Substitution request/assign screen for admin Teacher-facing 'today's schedule' view	Timetable table (period, day, subject, teacher, class, section) Substitution table + conflict-detection logic (no teacher double-booked) Notify substitute teacher and affected class on assignment

Phase 4 — Leave / Gate Pass (Weeks 12–13)

	Engineer A — Frontend	Engineer B — Backend
Deliverables	Leave application form (staff and student/parent-initiated) Approval queue for HOD/Principal (reuses the Admissions approval-dashboard pattern) Gate Pass request screen + printable/QR pass for the security desk	Leave request table + approval state machine (reused from the Admissions approval-step service, not rebuilt) Gate Pass table with time-stamped in/out logging Auto-notify parent when a Gate Pass is issued for their child

Phase 5 — Discipline & Safeguarding (Weeks 14–15)

Access control note

Safeguarding records are the most sensitive data in the system. This module needs stricter RBAC than the rest of the app — visibility limited to Discipline/Safeguarding roles and the Principal, plus an audit log of who viewed or edited each record. Build this as an explicit access layer, not an extension of the general permission system.

	Engineer A — Frontend	Engineer B — Backend
Deliverables	Incident logging form, restricted to authorized roles Case tracking dashboard with status (Open/Under Review/Closed)	Incident table with row-level access control beyond standard school_id scoping Audit log of every read/write to a record Escalation workflow to Principal for serious cases

	Confidential notes UI that visibly reflects access restrictions	

Phase 6 — Academics (Weeks 16–18)

	Engineer A — Frontend	Engineer B — Backend
Deliverables	Subject/curriculum setup screen (AO/Academic Officer) Gradebook / marks entry UI for teachers Report card preview and generation trigger Syllabus progress tracking view	Subject, Curriculum, and Marks tables Grade calculation rules engine (weighted averages, grade bands — configurable per school) Report card PDF generation, reusing the PDF service built for Admission Letters

Phase 7 — Reports & Dashboards (Weeks 19–20)

	Engineer A — Frontend	Engineer B — Backend
Deliverables	Admin/Principal analytics dashboard: attendance trends, fee collection status, academic performance, filterable by class/section/date range Export-to-Excel/PDF actions on every report view	Aggregation endpoints — likely materialized views or a scheduled aggregation job rather than live joins, to keep multi-tenant dashboards fast Excel/PDF export generation Cache aggregate results in Redis (ties back to Section 2.4) since these are read-heavy and don't need to be real-time to the second

Phase 8 — Hostel Management (Weeks 21–22)

	Engineer A — Frontend	Engineer B — Backend
Deliverables	Room/bed allocation screen Hostel roll-call / attendance UI Visitor log entry screen for hostel front desk	Hostel, Room, and Bed tables with capacity tracking Allocation logic (manual + auto-assign by availability) Roll-call API and visitor log table

Phase 9 — Staff Management & Payroll (Weeks 23–25)

	Engineer A — Frontend	Engineer B — Backend
Staff Management	Staff profile management screens (HR/Admin) Staff directory with role/department filters	Staff table extending the base User model Role/department schema, linked to the RBAC system from Phase 0
Payroll	Payroll processing screen for Accounts/HR	Salary structure + deduction rules engine

	Payslip view for individual staff	Pulls Leave balances from Phase 4 for unpaid-leave deductions Payslip PDF generation

Phase 10 — CBT: Computer-Based Testing (Weeks 26–27)

	Engineer A — Frontend	Engineer B — Backend
Deliverables	Test creation UI for teachers (question bank, MCQ + subjective) Student test-taking interface — timed, auto-submit on timeout Results view for teachers and students	Question bank and Test/Exam schema Timed-session handling (server-authoritative timer, not client-trusted) Auto-grading for objective questions; manual-grading queue for subjective ones

Phase 11 — Final QA, Integration & Deployment (Week 28)

	Engineer A — Frontend	Engineer B — Backend
Deliverables	Full regression pass across all 14 modules, every role Cross-module navigation and permission checks (e.g. a Parent can never reach a Staff-only screen)	Load test under simulated multi-school concurrent traffic Backup/disaster-recovery plan for Postgres Production deployment + monitoring/alerting across all modules
Joint sync point	Run one full school year's worth of scenarios end to end — admission through report cards, payroll, and a CBT exam — across two simulated schools simultaneously, to confirm tenant isolation actually holds.	

6. Full Timeline Summary

Phase	Module(s)
0	Foundations (multi-tenant schema, auth, infra)
1	Admissions
2	Student Information System, Parent Communication, Fees & Payments
3	Attendance, Timetable with Substitution
4	Leave / Gate Pass
5	Discipline & Safeguarding
6	Academics
7	Reports & Dashboards
8	Hostel Management
9	Staff Management, Payroll

10	CBT (Computer-Based Testing)
11	Final QA, Integration & Deployment

Total: ~28 weeks (roughly 6.5 months) for two engineers to cover all 14 modules at MVP-grade depth. This is a planning estimate, not a commitment — recalibrate after Phases 0–1 based on actual velocity, and use the cut order in Section 3 if the timeline needs to compress.

7. Appendix — Shared Status Enum (Reference for Both Engineers)

Status	Set by
Pending Approval	System, on AO 'Send for Approval'
Published	OS, on Approve
Submitted by Parent	System, on Parent Submit
Document Validated / Re-submit	Admission Incharge / AO
Payment Received	System, via payment gateway webhook
Fee Approved	Accounts, on Verify & Approve
Roll No Assigned	OS
VP Approved	Vice Principal
Admission Confirmed	Principal, final digital sign
Admission Complete	System, on Student Master write